GooDoc

Gestion de rendez-vous medicaux en ligne

Rapport de Projet

Licence Informatique — Institut Galilee

Annee universitaire 2024–2025

Rôle	Nom
Product Owner	PIERRE Jean-Samuel
Scrum Master	DAO Trung-Hieu
Developpeur Backend	CHAUVET Darren
Developpeur Frontend	COULIBALY Bourama

Methode agile — 4 sprints

Avril 2025

Resume

GooDoc est une plateforme web de gestion de rendez-vous medicaux developpee dans le cadre du cours de Licence Informatique a l'Institut Galilee. Elle permet a des patients de prendre des rendez-vous en ligne auprès de medecins, d'acceder a leur dossier medical electronique, de rejoindre des consultations video et de telecharger leurs ordonnances au format PDF.

Le projet a ete realise par une equipe de quatre etudiants selon la methodologie agile Scrum, decoupee en quatre sprints successifs. La stack technique retenue repose sur un frontend HTML/CSS/JavaScript (vanilla) servi par PHP pour la gestion de l'authentification, un backend Node.js/Express exposant une API REST, une base de donnees PostgreSQL, et l'integration de Jitsi Meet pour la visioconference.

Mots-cles : rendez-vous medical, application web, Node.js, PostgreSQL, API REST, methode agile, Scrum, dossier medical electronique, teleconsultation.

Ce document est confidentiel et destine a un usage academique uniquement.

Table des matières

Resume **1**

Liste des figures **4**

Liste des tableaux **5**

1 Presentation du projet **6**
- 1.1 Contexte et motivation . . . 6
- 1.2 Objectifs . . . 6
- 1.3 Perimètre fonctionnel . . . 6

2 Organisation de l'equipe et methode agile **8**
- 2.1 Composition de l'equipe . . . 8
- 2.2 Methodologie agile — Scrum . . . 8
- 2.3 Deroulement des quatre sprints . . . 9
- 2.4 Backlog produit — Principales user stories . . . 11

3 Architecture technique **12**
- 3.1 Vue d'ensemble . . . 12
- 3.2 Stack technique . . . 13
- 3.3 Structure du depôt . . . 13
- 3.4 Schema de la base de donnees . . . 14
- 3.5 API REST — Endpoints principaux . . . 15
- 3.6 Mecanisme d'authentification . . . 15

4 Conception UML **17**
- 4.1 Diagramme de cas d'utilisation . . . 17
- 4.2 Diagramme de classes . . . 17
- 4.3 Diagramme de sequence — Prise de rendez-vous . . . 17
- 4.4 Diagramme d'etats — RendezVous . . . 18
- 4.5 Diagramme d'activite — Flux complet . . . 18

5 Fonctionnalites detaillees **19**
- 5.1 Espace patient . . . 19
  - 5.1.1 Tableau de bord . . . 19

5.1.2 Gestion des rendez-vous 19
5.1.3 Dossier medical 19
5.1.4 Profil utilisateur 19
5.2 Espace medecin 20
5.3 Generation d'ordonnances PDF 20
5.4 Teleconsultation via Jitsi Meet 20
5.5 Notifications 20

6 Securite **21**
6.1 Authentification et autorisation 21
6.2 Protection des donnees 21

7 Installation et deploiement **22**
7.1 Prerequis 22
7.2 Procedure d'installation 22
7.3 Lancement 22

8 Bilan et perspectives **24**
8.1 Bilan du projet 24
8.2 Perspectives d'evolution 25

Conclusion **26**

A Variables d'environnement **27**

B Schema SQL simplifie **28**

C Repartition du travail **29**

Table des figures

1.1 Tableau de bord patient — GooDoc 7

3.1 Architecture en couches de GooDoc 12
3.2 Espace medecin — Tableau de bord avec liste des RDV et actions . . . 16
3.3 Dossier medical patient — Antecedents, profil et historique d'ordonnances 16

4.1 Diagramme d'etats du RendezVous 18

Liste des tableaux

2.1 Membres de l'equipe et rôles 8
2.2 Extrait du backlog produit 11

3.1 Stack technique retenue 13
3.2 Tables de la base de donnees GooDoc 14
3.3 Principaux endpoints de l'API GooDoc 15

8.1 Bilan de velocite par sprint 24

C.1 Repartition des tâches par membre 29

1. Presentation du projet

1.1 Contexte et motivation

La prise de rendez-vous medicaux reste souvent fastidieuse : appels telephoniques, files d'attente, manque de visibilite sur les disponibilites des praticiens. Ce constat nous a conduits a concevoir **GooDoc**, une plateforme web complète permettant de centraliser l'ensemble du parcours de soin numerique : de la reservation d'un creneau a la delivrance d'une ordonnance en passant par une consultation video.

Ce projet s'inscrit dans le cadre de notre formation en Licence Informatique a l'Institut Galilee. Il constitue un projet de fin d'annee visant a mettre en pratique les competences acquises en developpement web fullstack, conception de bases de donnees relationnelles et gestion de projet agile.

1.2 Objectifs

- Permettre a un patient de s'inscrire, de rechercher un medecin et de reserver un creneau disponible.
- Permettre a un medecin de gerer ses disponibilites, de valider ou refuser des rendez-vous et de demarrer une consultation video.
- Offrir un dossier medical electronique avec antecedents, allergies et historique d'ordonnances.
- Generer des ordonnances au format PDF telechargeable.
- Envoyer des notifications par e-mail aux deux parties.
- Integrer un service de visioconference via Jitsi Meet.

1.3 Perimètre fonctionnel

La figure 1.1 resume les grandes fonctionnalites du système.

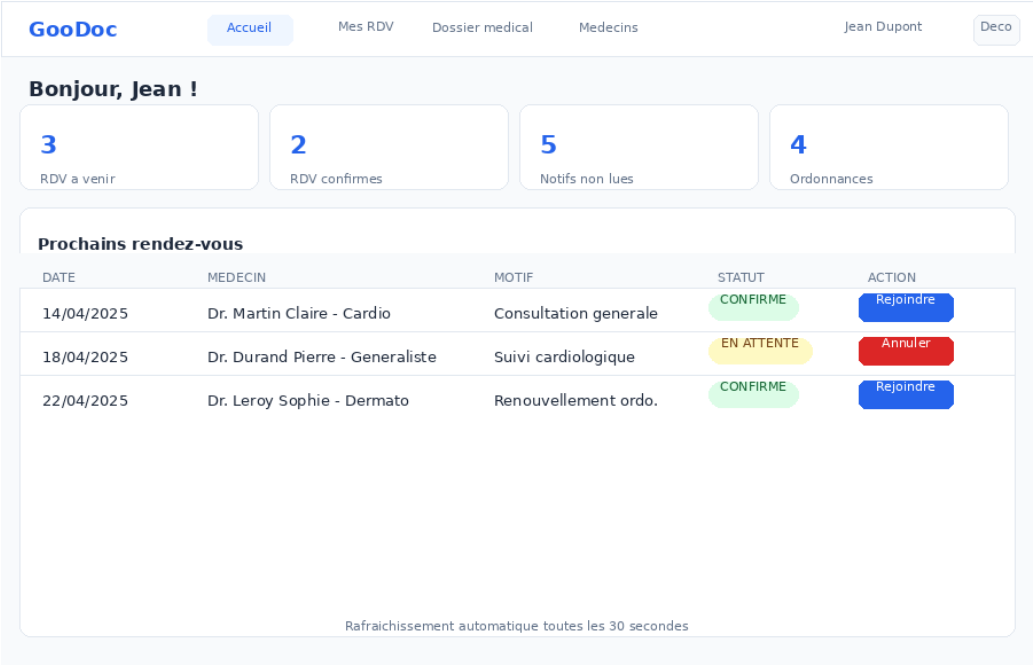

FIGURE 1.1 – Tableau de bord patient — GooDoc

2. Organisation de l'equipe et methode agile

2.1 Composition de l'equipe

L'equipe projet est constituee de quatre membres, chacun portant un rôle Scrum distinct.

Table 2.1 – Membres de l'equipe et rôles

Nom	Rôle	Responsabilites principales
PIERRE Jean-Samuel	Product Owner	Vision produit, priorisation du backlog, validation des livrables
DAO Trung-Hieu	Scrum Master	Animation des ceremonies Scrum, levee des obstacles, suivi de velocite
CHAUVET Darren	Developpeur Backend	API REST Node.js/Express, base de donnees PostgreSQL, JWT, PDF
COULIBALY Bourama	Developpeur Frontend	Interface HTML/CSS/JS, integration API, experience utilisateur

2.2 Methodologie agile — Scrum

Le projet a ete conduit selon la methodologie **Scrum**, cadre agile de developpement iteratif et incremental. Chaque sprint dure deux semaines et donne lieu aux ceremonies suivantes :

- **Sprint Planning** : selection des user stories a realiser dans le sprint.
- **Daily Stand-up** : point quotidien de 15 minutes sur l'avancement et les blocages.
- **Sprint Review** : demonstration des fonctionnalites livrees aux parties prenantes.
- **Sprint Retrospective** : analyse des axes d'amelioration du processus.

2.3 Deroulement des quatre sprints

Le projet s'est deroule en **quatre sprints** successifs, couvrant l'ensemble du cycle de developpement : de la mise en place de l'infrastructure jusqu'a la finalisation et la correction des bugs.

Sprint 1 — Infrastructure et authentification

Sprint 1 · Semaines 1–2

Objectif : Mettre en place les fondations du projet.

- Initialisation du depôt Git et de la structure de dossiers.
- Conception et creation du schema PostgreSQL (`table.sql`).
- Mise en place du serveur Node.js/Express avec gestion CORS.
- Implementation de l'authentification JWT (inscription, connexion, middleware).
- Pages PHP `login.php` et `register.php` avec session et mint JWT.
- Mise en place du client HTTP (`api.js`).

Sprint 2 — Gestion des rendez-vous et creneaux

Sprint 2 · Semaines 3–4

Objectif : Implementer le cœur metier de la plateforme.

- API de gestion des medecins et de leurs creneaux (`medecin.js`).
- API de gestion des rendez-vous (`rdv.js`) : creation, validation, annulation.
- Mise a jour automatique des statuts expires.
- Pages frontend : tableau de bord patient, liste des RDV, formulaire de reservation.
- Espace medecin : tableau de bord avec onglets RDV / Creneaux.
- Filtrage et recherche de medecins par specialite (`medecins.html`).

Sprint 3 — Dossier medical, ordonnances et visioconference

Sprint 3 · Semaines 5–6

Objectif : Enrichir le parcours de soin avec le dossier medical et la teleconsultation.

- API de dossier medical et d'ordonnances (`dossier.js`) : CRUD et generation PDF (PDFKit).
- Integration Jitsi Meet : generation de salon unique, API consultation video (`consultation.js`).
- Page dossier medical patient avec historique d'ordonnances et export PDF.
- Onglet ordonnances dans l'espace medecin avec modal de redaction.
- Service de notifications par e-mail via Nodemailer (`notification.js`).

Sprint 4 — Finalisation, profil utilisateur et qualite

Sprint 4 · Semaines 7–8

Objectif : Peaufiner l'experience utilisateur et corriger les anomalies.

- Page profil utilisateur (`profil.html`) avec modification des donnees personnelles et medicales.
- Système de toast de notifications visuelles en frontend.
- Rafraîchissement automatique des tableaux de bord toutes les 30 secondes.
- Correction de bugs : ordre des routes Express, gestion des creneaux sans date, tokens manquants.
- Redaction de la documentation technique (`README.md`).
- Creation des diagrammes UML : classes, sequence, activite, etats, cas d'utilisation.

2.4 Backlog produit — Principales user stories

Table 2.2 – Extrait du backlog produit

ID	Sprint	User Story	Points	Statut
US-01	1	En tant que patient, je veux m'inscrire et me connecter.	5	
US-02	1	En tant que medecin, je veux m'inscrire avec ma specialite.	3	
US-03	2	En tant que patient, je veux rechercher un medecin par specialite.	3	
US-04	2	En tant que patient, je veux reserver un creneau disponible.	8	
US-05	2	En tant que medecin, je veux valider ou refuser une demande de RDV.	5	
US-06	2	En tant que medecin, je veux gerer mes creneaux de disponibilite.	5	
US-07	3	En tant que patient, je veux consulter mon dossier medical.	5	
US-08	3	En tant que medecin, je veux rediger une ordonnance en consultation.	8	
US-09	3	En tant que patient, je veux telecharger mon ordonnance en PDF.	5	
US-10	3	En tant qu'utilisateur, je veux rejoindre une consultation video.	8	
US-11	3	En tant qu'utilisateur, je veux recevoir des notifications par e-mail.	5	
US-12	4	En tant qu'utilisateur, je veux modifier mon profil.	5	

3. Architecture technique

3.1 Vue d'ensemble

GooDoc repose sur une architecture en couches separant clairement les responsabilites : presentation, authentification, logique metier, persistance et services externes.

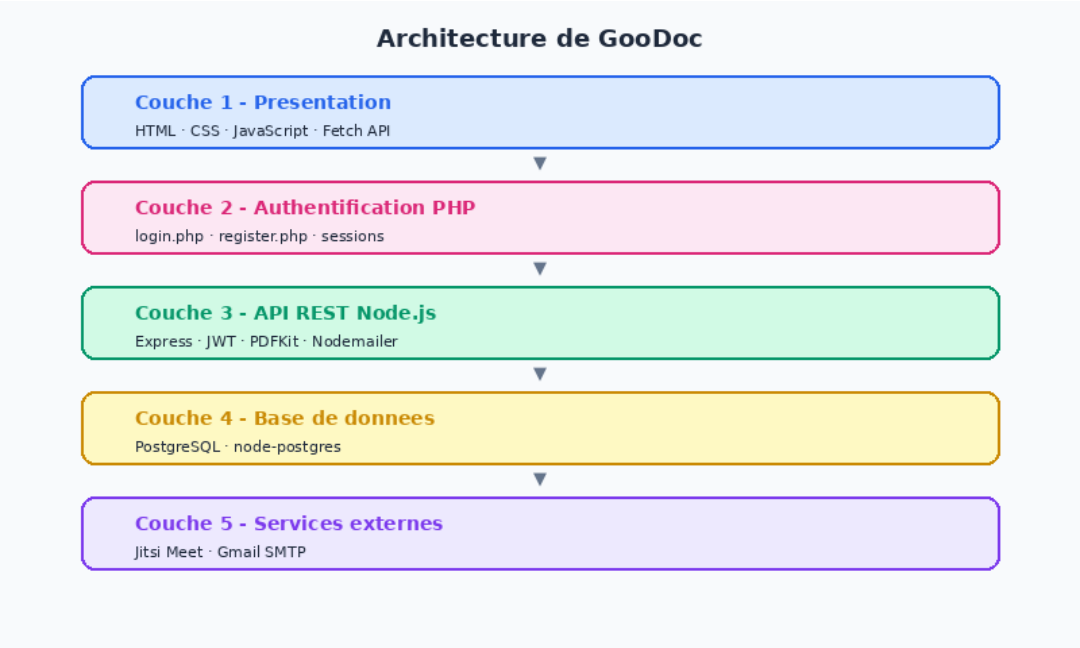

FIGURE 3.1 – Architecture en couches de GooDoc

3.2 Stack technique

Table 3.1 – Stack technique retenue

Couche	Technologies
Frontend	HTML5, CSS3, JavaScript (vanilla), Fetch API
Auth / Sessions	PHP $\geq$ 8.0, PDO, `password_hash()`, `$_SESSION`
API Backend	Node.js $\geq$ 18, Express 4, JWT (`jsonwebtoken`), `bcrypt`
Generation PDF	PDFKit 0.14
E-mails	Nodemailer 6
Base de donnees	PostgreSQL $\geq$ 14, `pg` (node-postgres)
Visioconference	Jitsi Meet (iframe / API)
Outillage dev	nodemon, npm, php -S

3.3 Structure du depôt

Listing 3.1 – Structure du projet GooDoc

```
gooddoc/
          backend/
                routes/
                      auth.js          # Authentification JWT
                      patient.js       # Profil patient
                      medecin.js       # Creneaux et donnees medecin
                      rdv.js           # Gestion des rendez-vous
                      dossier.js       # Dossier medical + PDF
                      consultation.js  # Sessions video Jitsi
                      notification.js  # Notifications + e-mails
                middleware/
                      auth.js          # Verification JWT
                db.js                # Pool PostgreSQL
                server.js            # Point d'entree Express
          frontend/
                pages/               # dashboard, rdv, dossier, medecin, profil
```

```
                js/                     # api.js, dashboard.js, rdv.js, ...
                css/style.css
                login.php / register.php / logout.php
                config.php              # Connexion PDO
            table.sql                   # Schema PostgreSQL
            README.md
```

3.4 Schema de la base de donnees

La base de donnees comprend neuf tables organisees autour de l'entite centrale `rendez_vous`.

Table 3.2 – Tables de la base de donnees GooDoc

Table	Description
`utilisateur`	Compte commun patient / medecin (nom, email, mot de passe hashe, rôle)
`patient`	Donnees medicales du patient (SS, groupe sanguin, adresse, naissance)
`medecin`	Donnees professionnelles (specialite, cabinet, tarif)
`dossier_medical`	Antecedents et allergies lies a un patient
`creneau`	Plage horaire de disponibilite d'un medecin (date, heure, statut)
`rendez_vous`	Reservation liant patient, medecin et creneau (statut ENUM)
`consultation_video`	URL Jitsi et statut de la session associee a un RDV
`ordonnance`	Medicaments et instructions prescrits, lies au dossier et au RDV
`notification`	Messages envoyes aux utilisateurs (type ENUM, statut lu/non lu)

3.5 API REST — Endpoints principaux

Table 3.3 – Principaux endpoints de l'API GooDoc

Methode	Endpoint	Description
POST	`/api/auth/token`	Connexion, retourne un JWT
POST	`/api/auth/mint`	Genère un JWT depuis PHP (secret interne)
GET	`/api/medecins`	Liste de tous les medecins
GET	`/api/medecins/:id/creneaux`	Creneaux disponibles d'un medecin
POST	`/api/rdv`	Creer un rendez-vous
PATCH	`/api/rdv/:id/statut`	Changer le statut d'un RDV
GET	`/api/dossier/:patientId`	Dossier medical + ordonnances
POST	`/api/dossier/ordonnance`	Creer une ordonnance
GET	`/api/dossier/ordonnance/:id/pdf`	Telecharger l'ordonnance PDF
POST	`/api/consultation`	Creer une session video Jitsi
GET	`/api/consultation/rdv/:rdvId`	Recuperer la session d'un RDV

3.6 Mecanisme d'authentification

L'authentification combine PHP pour la verification des identifiants et Node.js pour l'emission des tokens JWT.

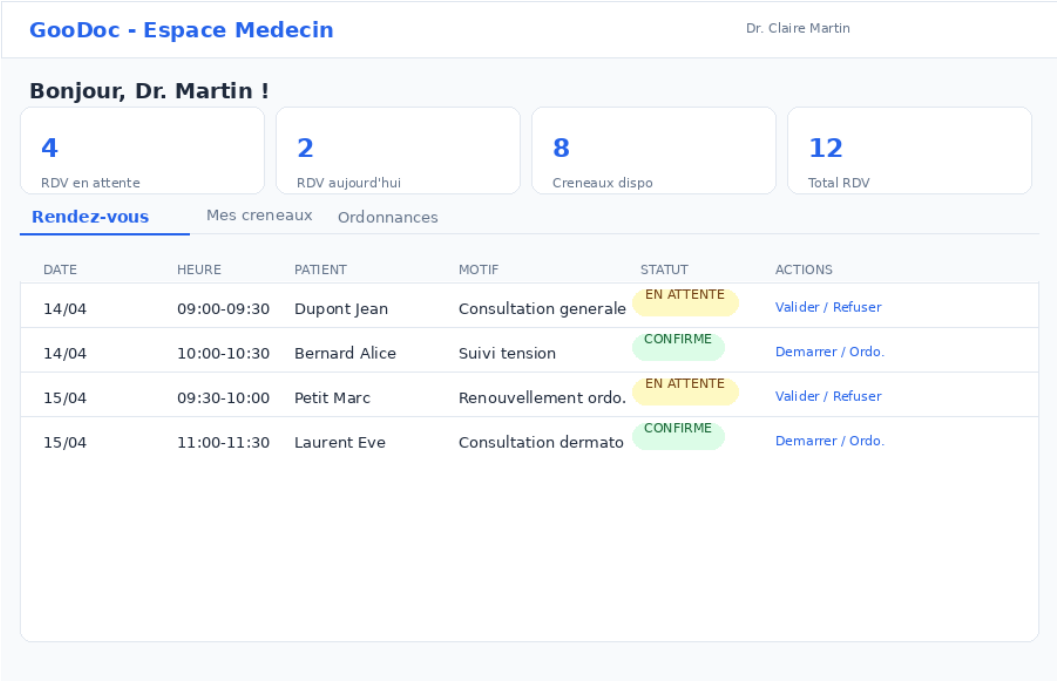

FIGURE 3.2 – Espace medecin — Tableau de bord avec liste des RDV et actions

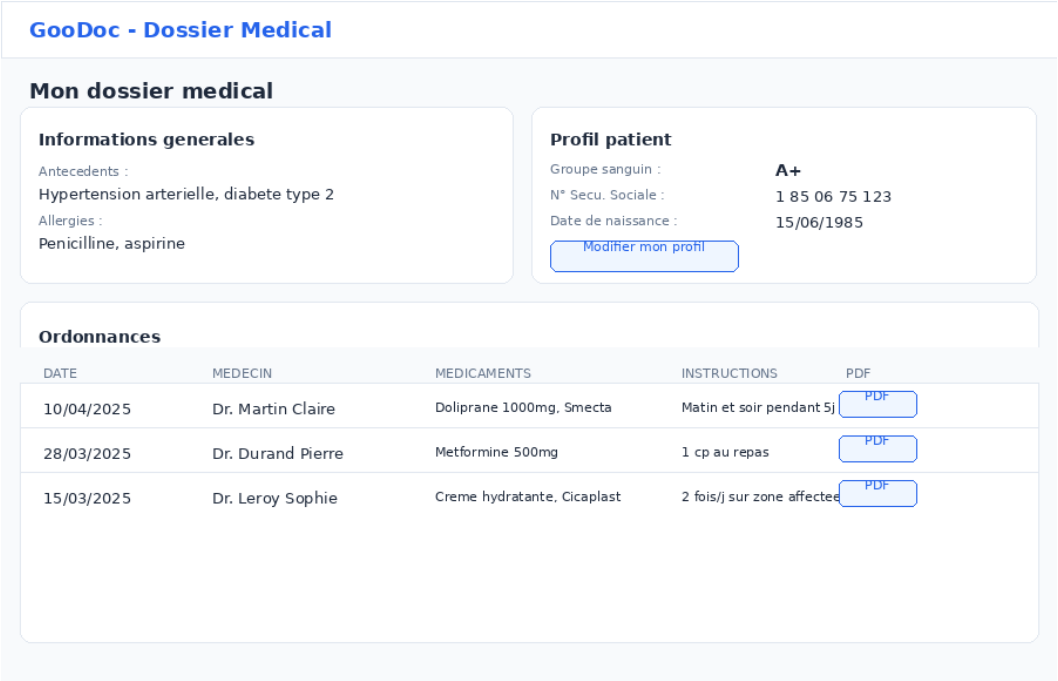

FIGURE 3.3 – Dossier medical patient — Antecedents, profil et historique d'ordonnances

4. Conception UML

4.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie deux acteurs principaux — le **Patient** et le **Medecin** — ainsi qu'un acteur secondaire de type système — le **Service Notifications**.

Acteurs principaux

- **Patient** : s'inscrire, se connecter, rechercher un medecin, reserver/annuler un RDV, rejoindre une consultation video, consulter son dossier, voir ses ordonnances.
- **Medecin** : s'inscrire, gerer ses creneaux, valider/refuser des RDV, demarrer une consultation, rediger une ordonnance, consulter le dossier d'un patient.
- **Service Notifications** (système) : envoyer des e-mails de confirmation et de rappel.

Relations utilisees : `«include»` (dependance obligatoire) et `«extend»` (extension optionnelle).

4.2 Diagramme de classes

Le diagramme de classes met en evidence la hierarchie d'heritage entre `Utilisateur`, `Patient` et `Medecin`, ainsi que les associations entre les entites metier.

Les principales relations sont :

- `Patient` 1 — 0..* `RendezVous`
- `Medecin` 1 — 0..* `RendezVous`
- `Medecin` 1 — 0..* `Creneau`
- `RendezVous` 1 — 0..1 `ConsultationVideo`
- `Patient` 1 — 1 `DossierMedical`
- `DossierMedical` 1 — 0..* `Ordonnance`

4.3 Diagramme de sequence — Prise de rendez-vous

Le diagramme de sequence decrit le flux nominal de prise de rendez-vous :

1. Le patient recherche un medecin (GET `/medecins`).
2. Il selectionne un creneau et confirme (POST `/rdv`).
3. Le système enregistre le RDV en statut `EN_ATTENTE`.
4. Le service de notification informe le medecin par e-mail.
5. Le medecin valide (PATCH `/rdv/:id/statut`).
6. Le système genère un lien Jitsi et notifie le patient.
7. Patient et medecin rejoignent la session video.
8. Le medecin redige une ordonnance (POST `/dossier/ordonnance`).
9. Le patient telecharge l'ordonnance PDF.

4.4 Diagramme d'etats — RendezVous

Le cycle de vie d'un rendez-vous est modelise par l'automate d'etats suivant :

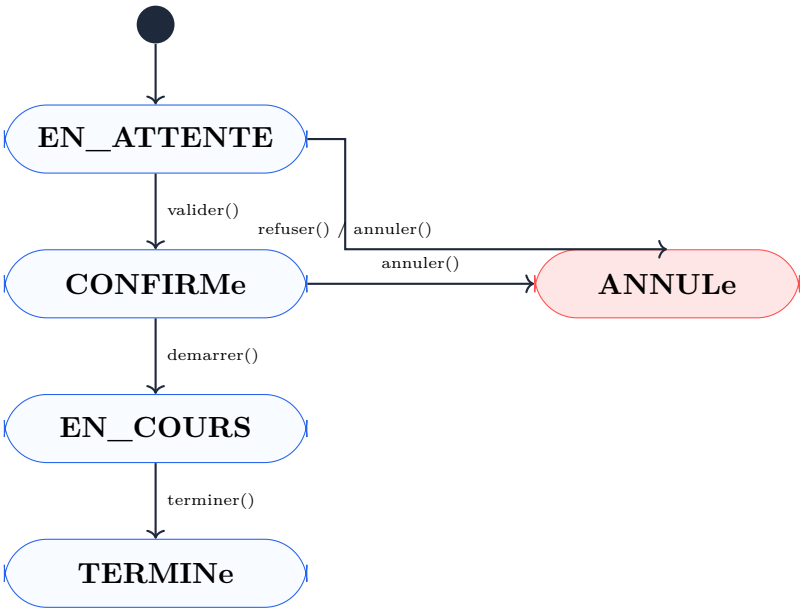

FIGURE 4.1 – Diagramme d'etats du RendezVous

4.5 Diagramme d'activite — Flux complet

Le diagramme d'activite modelise le flux de prise de rendez-vous et de consultation a travers trois couloirs (swim lanes) : **Patient**, **Système** et **Medecin**. Les activites principales sont : connexion, recherche, reservation, notification, validation, demarrage de la session video, redaction et telechargement de l'ordonnance.

5. Fonctionnalites detaillees

5.1 Espace patient

5.1.1 Tableau de bord

La page `dashboard.html` offre une vue consolidee : statistiques (RDV a venir, confirmes, notifications non lues, ordonnances), liste des cinq prochains rendez-vous avec actions contextuelles (rejoindre, annuler). Un rafraîchissement automatique toutes les 30 secondes maintient les donnees a jour.

5.1.2 Gestion des rendez-vous

La page `rdv.html` permet de :

- Consulter l'ensemble de ses rendez-vous actifs avec statuts colores.
- Ouvrir un modal de reservation : selection du medecin puis du creneau disponible.
- Annuler un rendez-vous en attente ou confirme.
- Rejoindre une consultation video Jitsi depuis le tableau.

5.1.3 Dossier medical

La page `dossier.html` affiche :

- Le resume du dossier : antecedents et allergies.
- Le profil medical : groupe sanguin, numero de securite sociale, date de naissance.
- L'historique des ordonnances avec export PDF individuel et suppression.

5.1.4 Profil utilisateur

La page `profil.html` permet la modification des donnees personnelles (nom, prenom, telephone, adresse) et medicales (groupe sanguin, numero SS). Un changement de mot de passe est egalement disponible. La sauvegarde met a jour la session `localStorage` pour un affichage coherent sans rechargement.

5.2 Espace medecin

L'interface medecin (`medecin.html`) est organisee en trois onglets :

Rendez-vous — Liste des RDV avec actions : valider, refuser, demarrer la consultation, rediger une ordonnance via un modal dedie.

Mes creneaux — Ajout de creneaux (date, heure debut, heure fin) et bascule disponible/bloque.

Ordonnances — Historique de toutes les ordonnances emises, avec suppression.

5.3 Generation d'ordonnances PDF

Lorsqu'un medecin valide une ordonnance, le backend genère un PDF via **PDFKit** contenant : en-tête GooDoc, nom du medecin, nom du patient, date, liste des medicaments et instructions. Le fichier est streame directement au navigateur avec l'en-tête `Content-Disposition: attachment`.

5.4 Teleconsultation via Jitsi Meet

Lors de la validation d'un RDV par le medecin, un salon Jitsi est automatiquement cree avec un identifiant unique de la forme `goodoc-rdv-{id}-{timestamp}`. L'URL `https://meet.jit.si/{salon}` est stockee en base et communiquee aux deux parties. Le statut de la consultation evolue de `EN_ATTENTE` a `EN_COURS` puis `TERMINEE`.

5.5 Notifications

Le service de notifications (`notification.js`) enregistre chaque notification en base de donnees et peut optionnellement envoyer un e-mail via **Nodemailer** (transport Gmail). Les types de notifications disponibles sont : `RAPPEL`, `CONFIRMATION`, `ANNULATION` et `INFO`.

6. Securite

6.1 Authentification et autorisation

- **Hachage des mots de passe** : `password_hash()` (bcrypt, PHP) et `bcrypt` (Node.js) avec un facteur de travail adapte.
- **JWT** : token signe avec un secret stocke en variable d'environnement, expiration configurable (defaut : 24h). Le middleware `auth.js` verifie la signature a chaque appel d'API protege.
- **Secret interne** : l'endpoint `/api/auth/mint` est protege par l'en-tête `X-Internal-Secret`, empêchant son accès depuis le navigateur.
- **CORS** : l'API n'autorise que les origines `localhost` et `localhost:8000`.

6.2 Protection des donnees

- Toutes les requêtes SQL utilisent des paramètres prepares (injection SQL impossible).
- Les variables d'environnement sensibles (`DB_PASSWORD`, `JWT_SECRET`, `MAIL_PASS`) sont stockees dans `.env`, exclu du depôt par `.gitignore`.
- Le fichier `frontend/.htaccess` est egalement exclu du depôt.

7. Installation et deploiement

7.1 Prerequis

- Node.js $\geq$ 18
- PHP $\geq$ 8.0 avec les extensions `pdo_pgsql` et `curl`
- PostgreSQL $\geq$ 14

7.2 Procedure d'installation

Listing 7.1 – Installation de GooDoc

```
# 1. Cloner le depot
git clone https://github.com/samuel-n04/gooddoc.git
cd gooddoc

# 2. Installer les dependances Node.js
cd backend && npm install

# 3. Configurer les variables d'environnement
cp backend/.env.example backend/.env
# Editer backend/.env avec vos valeurs

# 4. Creer la base de donnees et appliquer le schema
createdb goodoc
psql -U postgres -d goodoc -f table.sql
```

7.3 Lancement

Listing 7.2 – Demarrage des deux serveurs

```
# Terminal 1      API Node.js (port 3000)
cd backend && node server.js

# Terminal 2      Frontend PHP (port 8000)
cd frontend && php -S localhost:8000

```

```
# Ouvrir http://localhost:8000 dans le navigateur
```

8. Bilan et perspectives

8.1 Bilan du projet

Le projet GooDoc a ete mene a bien dans les delais impartis. Les quatre sprints ont permis de livrer l'integralite des fonctionnalites prevues au backlog initial, soit 12 user stories representant 65 points de story.

Table 8.1 – Bilan de velocite par sprint

Sprint	US realisees	Points livres	Statut
Sprint 1	2	8	Termine
Sprint 2	4	21	Termine
Sprint 3	5	31	Termine
Sprint 4	1	5	Termine
Total	**12**	**65**	**100%**

Points forts

- Architecture claire avec separation stricte frontend / backend / base de donnees.
- Authentification securisee combinant PHP (sessions) et JWT (API).
- Generation de PDF côte serveur sans dependance externe lourde.
- Integration Jitsi Meet sans infrastructure de signalisation propre.
- Interface responsive avec tableau de bord en temps quasi-reel.

Points d'amelioration

- L'utilisation de PHP uniquement pour les sessions introduit une dependance technologique supplementaire ; une migration vers une authentification entièrement Node.js serait plus coherente.
- L'absence de tests automatises (unitaires et d'integration) constitue un risque de regression lors des evolutions futures.
- Le frontend vanilla, bien que fonctionnel, gagnerait a être reecrit avec un framework (React, Vue) pour une meilleure maintenabilite a grande echelle.

8.2 Perspectives d'evolution

- **Application mobile** : portage des interfaces en React Native pour iOS et Android.
- **Paiement en ligne** : integration Stripe pour le règlement des consultations.
- **Intelligence artificielle** : suggestion de creneaux optimaux, analyse des symptômes pour orienter vers la bonne specialite.
- **Agenda intelligent** : synchronisation avec Google Calendar / Outlook.
- **Telemetrie et monitoring** : tableaux de bord d'exploitation (Grafana, Prometheus).
- **Tests automatises** : mise en place d'une suite Jest (backend) et Playwright (frontend).
- **Deploiement cloud** : containerisation Docker et deploiement sur AWS ou Azure.

Conclusion

Ce projet nous a permis de concevoir et developper une application web medicale complète en mettant en œuvre des competences complementaires : conception de bases de donnees relationnelles, developpement d'une API REST securisee, integration de services tiers et gestion de projet agile.

La methodologie Scrum, avec ses quatre sprints, s'est averee parfaitement adaptee : elle nous a permis de livrer des fonctionnalites utilisables dès la fin du premier sprint et de prioriser efficacement face aux contraintes de temps.

Sur le plan humain, le projet a renforce notre capacite a travailler en equipe avec des rôles bien definis, a communiquer regulièrement sur l'avancement et a faire preuve d'adaptation lorsque des difficultes techniques sont survenues.

GooDoc constitue une base solide pour une plateforme de sante numerique. Les nombreuses perspectives d'evolution identifiees temoignent du potentiel de l'application et de l'interêt de poursuivre son developpement dans un cadre professionnel.

A. Variables d'environnement

Listing A.1 – Fichier backend/.env.example

```
PORT=3000

# PostgreSQL
DB_HOST=localhost
DB_PORT=5432
DB_NAME=goodoc
DB_USER=postgres
DB_PASSWORD=

# JWT
JWT_SECRET=
JWT_EXPIRES_IN=24h

# Email (optionnel)
MAIL_USER=
MAIL_PASS=
```

B. Schema SQL simplifie

Listing B.1 – Extrait du schema PostgreSQL

```sql
CREATE TYPE statut_rdv AS ENUM
  ('EN_ATTENTE', 'CONFIRME', 'ANNULE', 'TERMINE');

CREATE TABLE utilisateur (
  id            SERIAL PRIMARY KEY,
  nom           VARCHAR(50)  NOT NULL,
  email         VARCHAR(100) NOT NULL UNIQUE,
  mot_de_passe VARCHAR(255) NOT NULL,
  role          VARCHAR(10)  NOT NULL CHECK (role IN ('patient',
     'medecin'))
);

CREATE TABLE rendez_vous (
  id          SERIAL PRIMARY KEY,
  patient_id INT NOT NULL REFERENCES patient(id),
  medecin_id INT NOT NULL REFERENCES medecin(id),
  creneau_id INT  REFERENCES creneau(id),
  date        DATE NOT NULL,
  motif       TEXT,
  statut      statut_rdv DEFAULT 'EN_ATTENTE',
  created_at TIMESTAMP DEFAULT NOW()
);
```

C. Repartition du travail

Table C.1 – Repartition des tâches par membre

Membre	Contributions principales
PIERRE Jean-Samuel	Definition du backlog, priorisation des US, maquettes fonctionnelles, validation des livrables, redaction du rapport et de la documentation.
DAO Trung-Hieu	Animation des sprints (planning, review, retrospective), suivi de la velocite, gestion du depôt Git, conception des diagrammes UML, coordination technique.
CHAUVET Darren	Conception du schema PostgreSQL, implementation de l'API Express (toutes les routes), middleware JWT, generation PDF (PDFKit), Nodemailer.
COULIBALY Bourama	Developpement de toutes les pages HTML/CSS, client JavaScript (`api.js`), integration des appels API, design de l'interface, authentification PHP.